

Ferhat Abbas Sétif 1 University
Faculty of Sciences
Department of Computer Science

Practical Work – Lab 2

Introduction to Deep Neural Networks

Prof. Abdelouahab Moussaoui

4th Year Engineering
Option: Artificial Intelligence
Academic Year: 2025-2026

> *"Theory without practice is empty,*
> *practice without theory is blind."*
>
> — Pr. Abdelouahab Moussaoui

The Deep Learning Journey

Learning by Doing: The Path to Mastery

This practical work is designed not just to be completed, but to be **understood deeply**. In the field of Deep Learning, as in all of computer science, there is a fundamental truth:

"You do not really understand something unless you can implement it yourself."

Why Practice is Essential:

- **Theoretical knowledge** alone gives you a map, but **practical experience** teaches you how to navigate the terrain.
- Reading about neural networks is not the same as building them, debugging them, and watching them learn.
- Every error message you encounter and resolve is a lesson that no textbook can teach.
- The challenges you face—overfitting, vanishing gradients, slow convergence—are best understood through direct experience.

Your Mission:

- **Do not just copy and paste** the code examples. Type them out manually, experiment with them, break them, and fix them.
- **Modify parameters** and observe what changes. Change the number of layers, the activation functions, the learning rate—and see what happens.
- **Ask "what if" questions** and answer them through experimentation.
- **Complete every exercise**—not just the ones that seem easy.
- For the mini-projects, **go beyond the minimum requirements**. Add your own ideas and improvements.

Remember:

- The difference between a beginner and an expert is not the number of tutorials they've read, but the number of problems they've solved.
- Every expert was once a beginner who refused to give up.
- The time you invest in truly understanding these concepts will pay dividends throughout your career.

The more you practice, the more you understand.
The more you understand, the more you can create.

"Knowledge is a treasure, but practice is the key to it."
— Thomas Fuller

"The man who removes a mountain begins by carrying away small stones."
— Chinese Proverb

— *Pr. Abdelouahab Moussaoui*

Objectives

Note: The objective of this practical session is to learn how to build, train, and evaluate artificial neural networks using **Keras** and **TensorFlow**.

- **Part 1**: Learning to use Keras through examples
- **Part 2**: Understanding deep learning concepts
- **Part 3**: Deep Learning in action on real-world problems
- **Part 4**: Mini-projects - Comprehensive application of all concepts

Important: **Prerequisites:** Python, basics of NumPy, Pandas, and scikit-learn.

Contents

The Deep Learning Journey **2**

Objectives **4**

1 Part 1: Learning to Use Keras **6**
1.1 Installation . . . 6
1.2 Exercise 1.1: Binary Classification with Keras . . . 6
1.3 Exercise 1.2: K-Fold Cross-Validation with Keras . . . 7
1.4 Exercise 1.3: Grid Search for Hyperparameter Optimization . . . 8

2 Part 2: Understanding Deep Learning Concepts **10**
2.1 Exercise 2.1: Activation Functions Comparison . . . 10
2.2 Exercise 2.2: Impact of Network Depth . . . 10
2.3 Exercise 2.3: Batch Size Experiments . . . 11
2.4 Exercise 2.4: Number of Epochs and Early Stopping . . . 11
2.5 Exercise 2.5: Optimizer Comparison . . . 12
2.6 Exercise 2.6: Learning Rate Impact . . . 12
2.7 Exercise 2.7: Regularization Techniques . . . 13
2.8 Exercise 2.8: Weight Initialization . . . 14

3 Part 3: Deep Learning in Action **15**
3.1 Exercise 3.1: Multi-class Classification - Iris Dataset . . . 15
3.2 Exercise 3.2: Binary Classification - Breast Cancer Wisconsin . . . 15
3.3 Exercise 3.3: Multi-class Classification - Wine Quality . . . 16
3.4 Exercise 3.4: Regression - Boston Housing . . . 16
3.5 Exercise 3.5: Binary Classification - Titanic Survival . . . 17
3.6 Exercise 3.6: Imbalanced Classification - Credit Card Fraud . . . 17
3.7 Exercise 3.7: Regression - California Housing . . . 18
3.8 Exercise 3.8: Feature Importance and Interpretability . . . 18

4 Part 4: Mini-Projects **20**
4.1 Mini-Project 1: Customer Churn Prediction System . . . 20
4.2 Mini-Project 2: House Price Prediction System . . . 23
4.3 Mini-Project 3: Air Quality Forecasting System . . . 27
4.4 Mini-Project 4: Energy Consumption Optimization System . . . 29
4.5 Mini-Project 5: Healthcare Readmission Prediction System . . . 31
4.6 Mini-Project Evaluation Rubric . . . 33

Summary of Datasets **34**

1 Part 1: Learning to Use Keras

1.1 Installation

First, install **TensorFlow** and **Keras**:

```
# Using pip
pip install tensorflow keras

# Or in Jupyter
!pip install tensorflow keras
```

Note: To verify installation: `import tensorflow; import keras`

1.2 Exercise 1.1: Binary Classification with Keras

Exercise 1.1: Binary Classification - Bank Customer Churn

Dataset: Bank Customer Churn Dataset (Kaggle) or Bank Marketing Dataset (UCI)
Study the following code that builds a neural network for predicting customer churn:

```
# Import libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
import tensorflow as tf
from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense

# Load dataset
dataset = pd.read_csv('Churn_Modelling.csv')
X = dataset.iloc[:, 3:13].values
y = dataset.iloc[:, 13].values

# Preprocess categorical variables
labelencoder_X_1 = LabelEncoder()
X[:, 1] = labelencoder_X_1.fit_transform(X[:, 1])
labelencoder_X_2 = LabelEncoder()
X[:, 2] = labelencoder_X_2.fit_transform(X[:, 2])

# One-hot encoding for country
ct = ColumnTransformer([("Geography", OneHotEncoder(), [1])],
    remainder='passthrough')
X = ct.fit_transform(X)
X = X[:, 1:]  # Avoid dummy variable trap

```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=0)

# Feature scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Build ANN
classifier = Sequential()
classifier.add(Dense(units=6, activation='relu', input_dim=11))
classifier.add(Dense(units=6, activation='relu'))
classifier.add(Dense(units=1, activation='sigmoid'))

# Compile
classifier.compile(optimizer='adam', loss='binary_crossentropy',
    metrics=['accuracy'])

# Train
classifier.fit(X_train, y_train, batch_size=10, epochs=100)

# Predict
y_pred = classifier.predict(X_test)
y_pred = (y_pred > 0.5)

# Evaluate
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

Question:

1. Write and execute the program above.
2. Add a Dropout layer after the second hidden layer: `classifier.add(Dropout(rate=0.2))`
3. Re-execute and observe the changes.
4. Modify the optimizer (try 'rmsprop'), batch_size (try 25, 32), and epochs (try 200).
5. What conclusions can you draw about the effect of these parameters?

1.3 Exercise 1.2: K-Fold Cross-Validation with Keras

Exercise 1.2: Cross-Validation for Neural Networks

Dataset: Churn Modelling Dataset (UCI)

```python
from keras.wrappers.scikit_learn import KerasClassifier
from sklearn.model_selection import cross_val_score

# Function to build the classifier
def build_classifier():
    classifier = Sequential()
    classifier.add(Dense(units=6, activation='relu', input_dim=11))
    classifier.add(Dense(units=6, activation='relu'))
    classifier.add(Dense(units=1, activation='sigmoid'))
    classifier.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return classifier

# Create classifier
classifier = KerasClassifier(build_fn=build_classifier, batch_size=10, epochs=100)

# Perform cross-validation
accuracies = cross_val_score(estimator=classifier, X=X_train, y=y_train, cv=10)

# Results
print("Mean accuracy:", accuracies.mean())
print("Standard deviation:", accuracies.std())
```

Question:

1. Write and execute the program.
2. Add dropout layers as in Exercise 1.1, then re-execute.
3. What conclusions can you draw about the effect of dropout on model stability?

1.4 Exercise 1.3: Grid Search for Hyperparameter Optimization

Exercise 1.3: Finding Best Hyperparameters

Dataset: Churn Modelling Dataset (UCI)

```python
from keras.wrappers.scikit_learn import KerasClassifier
from sklearn.model_selection import GridSearchCV

# Function with optimizer parameter
def build_classifier(optimizer):
    classifier = Sequential()
    classifier.add(Dense(units=6, activation='relu', input_dim=11))
    classifier.add(Dense(units=6, activation='relu'))
```

```
    classifier.add(Dense(units=1, activation='sigmoid'))
    classifier.compile(optimizer=optimizer, loss='
       binary_crossentropy', metrics=['accuracy'])
    return classifier

# Create classifier
classifier = KerasClassifier(build_fn=build_classifier)

# Define parameter grid
parameters = {'batch_size': [25, 32],
              'epochs': [100, 500],
              'optimizer': ['adam', 'rmsprop']}

# Grid search
grid_search = GridSearchCV(estimator=classifier,
                           param_grid=parameters,
                           scoring='accuracy',
                           cv=10)

grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best accuracy:", grid_search.best_score_)
```

Question:

1. Write and execute the program.
2. Add dropout layers, then re-execute.
3. How does dropout affect the optimal hyperparameters?

2 Part 2: Understanding Deep Learning Concepts

2.1 Exercise 2.1: Activation Functions Comparison

Objective: Compare different activation functions and understand their impact on network performance.

Dataset: Iris Dataset (UCI)

Question: Using the Iris dataset from UCI:

1. Build a neural network with one hidden layer (10 neurons).
2. Train the network with the following activation functions for the hidden layer:
   - Sigmoid
   - Tanh
   - ReLU
   - Leaky ReLU
3. For each activation, record:
   - Training accuracy
   - Validation accuracy
   - Number of epochs to converge
4. Plot the loss curves for all activations on the same graph.
5. Which activation function performs best? Why?
6. What problem does ReLU solve compared to sigmoid?

2.2 Exercise 2.2: Impact of Network Depth

Objective: Understand how the number of layers affects learning capability and overfitting.

Dataset: Wine Dataset (UCI)

Question: Using the Wine dataset from UCI:

1. Build networks with the following architectures (all with ReLU activation):
   - Shallow: [64]
   - Medium: [128, 64]
   - Deep: [256, 128, 64]
   - Very Deep: [512, 256, 128, 64]
2. Train each network for 100 epochs.

3. For each architecture, record:
   - Training accuracy
   - Validation accuracy
   - Training time
4. Plot learning curves for each architecture.
5. At what depth does overfitting begin to appear?
6. How can you mitigate overfitting in deep networks?

2.3 Exercise 2.3: Batch Size Experiments

Objective: Understand the impact of batch size on training dynamics.
Dataset: Breast Cancer Wisconsin Dataset (UCI)

Question: Using the Breast Cancer Wisconsin dataset:

1. Build a fixed architecture: [128, 64, 32] with ReLU activation.
2. Train with different batch sizes: 8, 16, 32, 64, 128, 256.
3. For each batch size, record:
   - Final accuracy
   - Training time per epoch
   - Loss curve stability
4. Create a table comparing results.
5. What is the optimal batch size? Why?
6. How does batch size affect convergence speed?

2.4 Exercise 2.4: Number of Epochs and Early Stopping

Objective: Learn to determine the optimal number of training epochs.
Dataset: Heart Disease Dataset (UCI)

Question: Using the Heart Disease dataset:

1. Build a neural network with architecture of your choice.
2. Train for 500 epochs without any regularization.
3. Plot training and validation loss on the same graph.
4. At what epoch does overfitting begin?

5. Implement early stopping with different patience values (5, 10, 20).
6. Compare the results with and without early stopping.
7. What is the optimal number of epochs for this dataset?

2.5 Exercise 2.5: Optimizer Comparison

Objective: Compare different optimization algorithms.
Dataset: MNIST Dataset (Yann LeCun's website)

Question: Using the MNIST dataset:

1. Build a network with 2 hidden layers (128 and 64 neurons).
2. Train with different optimizers using default parameters:
   - SGD
   - SGD with momentum
   - RMSprop
   - Adam
   - AdamW
   - Nadam
3. For each optimizer, record:
   - Final accuracy
   - Convergence speed (epochs to reach 90% of final accuracy)
   - Training time
4. Plot loss curves for all optimizers.
5. Which optimizer converges fastest? Which gives best accuracy?
6. Why does Adam generally outperform SGD?

2.6 Exercise 2.6: Learning Rate Impact

Objective: Understand the critical role of learning rate.
Dataset: Pima Indians Diabetes Dataset (UCI)

Question: Using the Pima Indians Diabetes dataset:

1. Build a simple network (1 hidden layer with 32 neurons).
2. Train with different learning rates: 1.0, 0.1, 0.01, 0.001, 0.0001, 0.00001.

3. For each learning rate, observe:
    - Does the network converge?
    - How many epochs to converge?
    - Final accuracy
4. Create a plot of accuracy vs learning rate.
5. What happens with very high learning rates (1.0)? Why?
6. What happens with very low learning rates (0.00001)? Why?
7. What is the optimal learning rate range for your network?

2.7 Exercise 2.7: Regularization Techniques

Objective: Compare different regularization methods.
Dataset: Spambase Dataset (UCI)

Question: Using the Spambase dataset:

1. Build an overparameterized network (multiple large layers).
2. Train a baseline model without regularization (observe overfitting).
3. Apply each regularization technique separately:
    - L1 regularization (try different lambda values)
    - L2 regularization (try different lambda values)
    - Dropout (try different rates: 0.2, 0.3, 0.5)
    - Batch Normalization
4. For each technique, record training and validation accuracy.
5. Create a table comparing results.
6. Which technique works best for your dataset?
7. Try combining techniques (e.g., Dropout + L2). Does it help?

2.8 Exercise 2.8: Weight Initialization

Objective: Understand the importance of proper weight initialization.
Dataset: Forest Covertype Dataset (UCI)

Question: Build a deep network with 5 hidden layers using sigmoid activation:

1. Try different initialization methods:
   - Random normal (mean=0, std=1)
   - Random normal (mean=0, std=0.01)
   - Xavier/Glorot initialization
   - He initialization
2. For each initialization, monitor:
   - Does the network converge?
   - Gradient norms at different layers
   - Final accuracy
3. Plot the gradient distribution for the first and last layers.
4. Which initialization works best? Why?
5. How does proper initialization help with the vanishing gradient problem?

3 Part 3: Deep Learning in Action

3.1 Exercise 3.1: Multi-class Classification - Iris Dataset

Dataset: Iris Dataset (UCI)

Question:

1. Load the Iris dataset from the UCI repository.
2. Perform necessary preprocessing (encoding of target variable, train-test split, feature scaling).
3. Propose an ANN architecture for classifying iris flowers into three species.
4. Train and evaluate your model.
5. Improve your architecture by adding hidden layers, changing activation functions, or adding regularization.
6. Compare your results with classical machine learning methods (SVM, Random Forest, Logistic Regression).
7. Draw conclusions about the performance of neural networks vs classical methods on this dataset.

3.2 Exercise 3.2: Binary Classification - Breast Cancer Wisconsin

Dataset: Breast Cancer Wisconsin (Diagnostic) Dataset (UCI)

Question:

1. Load the Breast Cancer Wisconsin dataset.
2. Explore the data: features, class distribution, statistics.
3. Build a neural network for binary classification (malignant vs benign).
4. Optimize your architecture using validation data.
5. Evaluate your model using accuracy, precision, recall, F1-score, and confusion matrix.
6. Compare with classical ML methods (KNN, SVM, Random Forest).
7. Which approach gives the best results? Why might neural networks be suitable for medical diagnosis?

3.3 Exercise 3.3: Multi-class Classification - Wine Quality

Dataset: Wine Quality Dataset (UCI)

Question:

1. Load the Wine Quality dataset (choose either red or white wine).
2. This can be treated as either regression (predict quality score) or classification (predict quality category).
3. Preprocess the data appropriately.
4. Build and train a neural network for your chosen task.
5. Experiment with different architectures and hyperparameters.
6. Evaluate your model using appropriate metrics (MSE/MAE for regression, accuracy/F1 for classification).
7. Compare with classical methods (Linear Regression/Random Forest Regressor for regression, or classification algorithms).
8. Discuss the strengths and weaknesses of neural networks for this type of problem.

3.4 Exercise 3.4: Regression - Boston Housing

Dataset: Boston Housing Dataset (CMU StatLib)

Question:

1. Load the Boston Housing dataset.
2. Explore the features and target variable (median house value).
3. Build a neural network for regression to predict housing prices.
4. Experiment with different architectures (number of layers, neurons per layer).
5. Use different activation functions in hidden layers.
6. Evaluate using MSE, MAE, and R^2 score.
7. Compare with classical regression methods (Linear Regression, Ridge, Lasso, Random Forest Regressor).
8. Which model performs best? Why might neural networks be overkill for this dataset?

3.5 Exercise 3.5: Binary Classification - Titanic Survival

Dataset: Titanic Dataset (Kaggle)

Question:

1. Load the Titanic dataset from Kaggle.
2. Perform comprehensive data preprocessing:
   - Handle missing values (Age, Cabin, Embarked)
   - Encode categorical variables (Sex, Embarked)
   - Create new features (FamilySize, Title from Name)
   - Scale numerical features
3. Build a neural network to predict survival.
4. Experiment with different architectures and regularization techniques.
5. Use cross-validation to tune hyperparameters.
6. Evaluate your final model on the test set.
7. Compare with classical ML methods (Logistic Regression, Random Forest, XGBoost).
8. Which features are most important for prediction? Can you interpret what the network learned?

3.6 Exercise 3.6: Imbalanced Classification - Credit Card Fraud

Dataset: Credit Card Fraud Detection Dataset (Kaggle)

Question:

1. Load the Credit Card Fraud dataset (highly imbalanced: 0.17% fraud).
2. Analyze the class distribution and its implications.
3. Build a baseline neural network without addressing imbalance.
4. Implement techniques to handle class imbalance:
   - Class weights in the loss function
   - Oversampling (SMOTE)
   - Undersampling
   - Focal Loss (custom loss function)
5. Evaluate using appropriate metrics (not accuracy!):

- Precision, Recall, F1-score for the minority class
- ROC-AUC
- Precision-Recall AUC
- Confusion Matrix

6. Compare the effectiveness of different imbalance handling techniques.
7. Which approach works best for fraud detection? Why is recall particularly important for this problem?

3.7 Exercise 3.7: Regression - California Housing

Dataset: California Housing Dataset

Question:

1. Load the California Housing dataset (available in scikit-learn: `sklearn.datasets.fetch_califor`
2. Explore the data and target distribution.
3. Build a neural network for regression.
4. Experiment with:
   - Different loss functions (MSE, MAE, Huber)
   - Different architectures
   - Batch normalization
   - Dropout
5. Use k-fold cross-validation to evaluate model stability.
6. Compare with Random Forest Regressor and Gradient Boosting.
7. Create scatter plots of predicted vs actual values.
8. Analyze the residuals. Is there any pattern?

3.8 Exercise 3.8: Feature Importance and Interpretability

Dataset: Adult Census Income Dataset (UCI)

Question:

1. Load the Adult dataset (predict whether income exceeds \$50K/year).
2. Preprocess the data (handle categorical variables, scale features).
3. Train an optimized neural network.

4. Implement feature importance analysis:
    - Permutation importance
    - Gradient-based importance (using weights)
    - SHAP values (use the SHAP library)
5. Compare the top important features from each method.
6. Extract features from the last hidden layer and use t-SNE to visualize in 2D.
7. Color points by true class. Are the classes separable?
8. What does this tell you about what the network learned?
9. Do the important features make sense from a domain perspective?

4 Part 4: Mini-Projects

Important: These mini-projects are designed to be comprehensive assignments that integrate all the concepts learned in this practical work. Students are expected to work in groups of 2-3 and submit a complete report including code, visualizations, and analysis.

4.1 Mini-Project 1: Customer Churn Prediction System

Dataset: Telco Customer Churn Dataset (Kaggle)

Question: **Context:** You are a data scientist at a telecommunications company. The marketing department wants to reduce customer churn (customers leaving for competitors). They have provided you with historical customer data and asked you to build a predictive model that can identify customers at high risk of churning.
Objective: Develop a complete customer churn prediction system using deep learning.
Dataset Description: The dataset contains 7043 customers with 21 features including:

- Customer demographics (gender, senior citizen, partner, dependents)
- Services signed up for (phone, multiple lines, internet, online security, etc.)
- Account information (tenure, contract type, payment method, monthly charges, total charges)
- Target variable: Churn (Yes/No)

Part A: Data Exploration and Preprocessing (20% of grade)

1. Load and explore the dataset. Check for missing values, data types, and basic statistics.
2. Perform exploratory data analysis (EDA):
   - Visualize the distribution of churn (is it balanced or imbalanced?)
   - Analyze the relationship between features and churn
   - Create correlation matrices and heatmaps
   - Identify patterns and insights
3. Handle missing values appropriately (TotalCharges column may have issues).
4. Encode categorical variables (use appropriate encoding techniques).
5. Scale numerical features.
6. Split the data into training (70%), validation (15%), and test (15%) sets.

Part B: Baseline Models (15% of grade)

1. Build and evaluate classical machine learning models:

- Logistic Regression
- Random Forest Classifier
- XGBoost Classifier

2. Use cross-validation to evaluate these models.
3. Record performance metrics: accuracy, precision, recall, F1-score, ROC-AUC.
4. Create confusion matrices for each model.

Part C: Neural Network Architecture Design (25% of grade)

1. Design and implement multiple neural network architectures:
   - Architecture 1: Simple network with 1-2 hidden layers
   - Architecture 2: Deeper network with 3-4 hidden layers
   - Architecture 3: Network with dropout and batch normalization
2. Experiment with different activation functions (ReLU, tanh, LeakyReLU).
3. Try different optimizers (Adam, RMSprop, SGD with momentum).
4. Implement early stopping to prevent overfitting.
5. Use validation data to tune hyperparameters.

Part D: Addressing Class Imbalance (15% of grade)

1. Analyze the class distribution. Is the dataset balanced?
2. If imbalanced, implement techniques to handle it:
   - Class weights in the loss function
   - Oversampling using SMOTE
   - Undersampling
   - Focal Loss implementation
3. Compare the effectiveness of these techniques.
4. Determine which approach gives the best recall for the churn class.

Part E: Hyperparameter Optimization (15% of grade)

1. Use GridSearchCV or RandomizedSearchCV to find optimal hyperparameters:
   - Number of layers and neurons
   - Learning rate
   - Batch size
   - Dropout rate

- L2 regularization parameter

2. Document your search process and results.
3. Train your final model with the best hyperparameters.

Part F: Model Evaluation and Interpretation (10% of grade)

1. Evaluate your final model on the test set using multiple metrics.
2. Create visualizations:
   - Confusion matrix heatmap
   - ROC curve with AUC
   - Precision-Recall curve
   - Learning curves (training/validation loss and accuracy)
3. Perform feature importance analysis:
   - Permutation importance
   - SHAP values for interpretability
4. Identify the top factors that influence churn.
5. Provide business recommendations based on your findings.

Part G: Report and Presentation (10% of grade)

1. Write a comprehensive report including:
   - Introduction and problem statement
   - Data exploration findings
   - Methodology (models, experiments)
   - Results and comparisons
   - Business recommendations
   - Limitations and future work
2. Prepare a 10-minute presentation summarizing your work.
3. Include visualizations and key insights.

Deliverables:

- Python code (Jupyter notebook or .py files)
- Final trained model (saved in .h5 format)
- Comprehensive report (PDF)
- Presentation slides

4.2 Mini-Project 2: House Price Prediction System

Dataset: House Prices: Advanced Regression Techniques (Kaggle)

Question: **Context:** You work for a real estate investment firm. Your company is considering purchasing properties in various neighborhoods and needs an accurate model to predict house prices. The firm has collected detailed data on house sales and wants you to build a deep learning model that can estimate the sale price of a house given its characteristics.
Objective: Develop a comprehensive house price prediction system using deep learning for regression.
Dataset Description: The dataset contains 1460 training examples and 1459 test examples with 79 features describing various aspects of residential homes. Features include:

- Property characteristics (lot size, zoning, shape, utilities)
- Building features (number of rooms, bedrooms, bathrooms, quality, condition)
- Basement and garage details
- Exterior features (roof, siding, porch)
- Amenities (fireplace, pool, central air)
- Location features (neighborhood, proximity to amenities)
- Target variable: SalePrice (continuous)

Part A: Data Exploration and Preprocessing (20% of grade)

1. Load and explore the dataset. Check for missing values (there will be many!).
2. Perform comprehensive EDA:
    - Analyze the distribution of SalePrice (is it normal? skewed?)
    - Create histograms for numerical features
    - Analyze categorical variables and their relationship with price
    - Create correlation matrices for numerical features
    - Identify outliers and decide how to handle them
3. Handle missing values strategically:
    - For numerical features: consider mean/median imputation or create indicator variables
    - For categorical features: consider mode imputation or "None" category
4. Feature engineering:
    - Create new features (TotalSF = 1stFlrSF + 2ndFlrSF + TotalBsmtSF)
    - Create interaction terms for important features

- Bin some continuous variables if appropriate

5. Encode categorical variables (consider target encoding for high-cardinality features).
6. Apply log transformation to SalePrice if it's skewed.
7. Scale numerical features.
8. Split data into training (70%), validation (15%), and test (15%).

Part B: Baseline Models (15% of grade)

1. Build and evaluate classical regression models:
   - Linear Regression (with regularization: Ridge, Lasso, ElasticNet)
   - Random Forest Regressor
   - Gradient Boosting Regressor (XGBoost, LightGBM)
2. Use cross-validation to evaluate these models.
3. Record performance metrics: RMSE, MAE, R^2 score.
4. Create scatter plots of predicted vs actual values.
5. Analyze residuals for patterns.

Part C: Neural Network Architecture Design (25% of grade)

1. Design and implement multiple neural network architectures:
   - Architecture 1: Simple network with 2-3 hidden layers
   - Architecture 2: Deeper network with 4-5 hidden layers
   - Architecture 3: Wide network with many neurons in first layer
   - Architecture 4: Network with residual connections
2. Experiment with different activation functions (ReLU, LeakyReLU, ELU).
3. Try different loss functions (MSE, MAE, Huber).
4. Implement regularization techniques:
   - L1/L2 regularization
   - Dropout
   - Batch Normalization
5. Use learning rate scheduling (step decay, exponential decay, ReduceLROnPlateau).
6. Implement early stopping with patience.

Part D: Feature Selection and Dimensionality Reduction (10% of grade)

1. Analyze feature importance from tree-based models.

2. Use correlation analysis to identify redundant features.
3. Consider dimensionality reduction techniques:
   - PCA (Principal Component Analysis)
   - Autoencoders for feature compression
4. Compare model performance with full features vs selected features.

Part E: Hyperparameter Optimization (15% of grade)

1. Use GridSearchCV, RandomizedSearchCV, or Bayesian Optimization:
   - Number of layers and neurons per layer
   - Learning rate (try different orders of magnitude)
   - Batch size (16, 32, 64, 128)
   - Dropout rate (0.1, 0.2, 0.3, 0.5)
   - L1/L2 regularization strength
   - Optimizer choice and parameters
2. Document your search strategy and results.
3. Train ensemble of best models (if applicable).

Part F: Model Evaluation and Interpretation (10% of grade)

1. Evaluate your final model on the test set:
   - RMSE, MAE, R^2 score
   - Scatter plot: predicted vs actual values
   - Residual plot (check for heteroscedasticity)
   - Distribution of errors
2. Compare your best neural network with the best classical model.
3. Create visualizations:
   - Learning curves (training/validation loss)
   - Feature importance (if possible)
   - Error analysis (which houses are hardest to predict?)
4. Interpret the model's predictions:
   - Use SHAP values to explain individual predictions
   - Identify which features increase/decrease house price
   - Provide insights for the real estate investment team

Part G: Ensemble and Advanced Techniques (5% of grade)

1. Try ensemble methods:
   - Averaging predictions from multiple neural networks
   - Stacking: neural network + XGBoost + Random Forest
2. Experiment with pseudo-labeling using test data.
3. If time permits, try k-fold cross-validation ensemble.

Part H: Report and Presentation (10% of grade)

1. Write a comprehensive report including:
   - Executive summary for business stakeholders
   - Technical methodology section
   - Data exploration findings and insights
   - Model development process and experiments
   - Results and comparisons
   - Feature importance and business implications
   - Limitations and future improvements
2. Prepare a 15-minute presentation with:
   - Problem overview
   - Key insights from data
   - Model comparison and results
   - Business recommendations
   - Demo of prediction system

Deliverables:

- Python code with clear documentation
- Final trained model(s)
- Comprehensive technical report (PDF)
- Business presentation (PPT/PDF)
- Simple web app or API for predictions (bonus)

4.3 Mini-Project 3: Air Quality Forecasting System

Dataset: Air Quality Dataset (UCI) or EPA Air Quality (Kaggle)

Context: Environmental protection agency needs air quality forecasts for public health warnings.

Objective: Multi-output neural network predicting multiple pollutants simultaneously.

Dataset: Hourly measurements of meteorological data and pollutants (CO, NO2, O3, PM10, PM2.5).

Part A: Time Series Preprocessing (20%)

- Handle datetime
- Analyze temporal patterns
- Create time features (hour, day, season)
- Lag features and rolling statistics
- Temporal train/val/test split

Part B: EDA (10%)

- Visualize pollutant patterns
- Correlations with meteorology
- Identify extreme events

Part C: Baseline Models (15%)

- Persistence model
- Seasonal naive
- Linear regression
- Random Forest per pollutant

Part D: Multi-Output Neural Networks (25%)

- Shared bottom architecture
- Tower architecture
- Attention mechanisms
- LSTM/GRU for sequences
- Experiment with loss weighting

Part E: Uncertainty Quantification (10%)

- Monte Carlo Dropout
- Deep Ensembles
- Quantile regression
- Prediction intervals

Part F: Alert System (10%)

- Define alert thresholds
- Calculate false alarm rate
- Design public health warnings

Part G: Report (10%)

- Executive summary for officials
- Technical methodology
- Recommendations

4.4 Mini-Project 4: Energy Consumption Optimization System

Dataset: Household Electric Power Consumption (UCI)

Context: Energy company wants to help customers reduce consumption and carbon footprint.

Objective: System combining prediction, anomaly detection, and clustering.

Dataset: 4 years of minute-level measurements of power consumption and sub-metering.

Part A: Preprocessing (15%)

- Handle missing values
- Resample to hourly/daily
- Create time features
- Rolling statistics

Part B: EDA (10%)

- Daily/weekly patterns
- Seasonal variations
- Appliance usage patterns
- Peak demand analysis

Part C: Task 1 - Forecasting (25%)

- ARIMA/SARIMA
- Prophet
- LSTM networks
- Sequence-to-sequence models
- Multi-step forecasting

Part D: Task 2 - Anomaly Detection (20%)

- Statistical methods (Z-score, IQR)
- Isolation Forest, One-Class SVM
- Autoencoders for reconstruction
- LSTM Autoencoders
- Different anomaly types

Part E: Task 3 - Customer Segmentation (15%)

- Extract consumption profiles
- K-Means, DBSCAN
- Deep Embedded Clustering
- Characterize clusters
- Energy-saving recommendations

Part F: Task 4 - Recommendations (10%)

- Rule-based recommendation engine
- Personalized tips
- Savings estimates

Part G: Dashboard (5%)

- Simple dashboard
- Current vs forecast
- Detected anomalies
- Recommendations

4.5 Mini-Project 5: Healthcare Readmission Prediction System

Dataset: Diabetes Readmission Dataset (UCI)

Context: Hospital network needs to identify patients at high risk of 30-day readmission for targeted interventions.

Objective: Healthcare analytics system predicting readmission risk and recommending interventions.

Dataset: 100,000+ records of diabetes patients from 130 hospitals. Features include demographics, admission details, diagnoses, procedures, medications.

Part A: Healthcare Data Preprocessing (15%)

- Handle missing values
- Process ICD-9 codes (map to categories)
- Create comorbidity scores
- Feature engineering (prior utilization, medication complexity)
- Handle class imbalance

Part B: Medical EDA (10%)

- Readmission rates by demographics
- Readmission by diagnosis
- Time to readmission analysis
- Identify key risk factors

Part C: Baseline Models (10%)

- LACE Index (clinical score)
- HOSPITAL Score
- Logistic Regression
- Random Forest, XGBoost
- Clinical metrics (sensitivity, PPV)

Part D: Neural Network Architectures (20%)

- Feed-forward networks
- Multi-input networks (demographic, clinical, medication)
- Attention mechanisms
- Survival analysis (DeepSurv)

- Weighted loss functions

Part E: Interpretability (15%)

- SHAP values for explanations
- Individual patient risk profiles
- Feature contribution plots
- Validate with clinical knowledge

Part F: Intervention Recommendations (10%)

- Risk stratification
- Personalized interventions
- Follow-up scheduling
- Care management programs

Part G: Fairness Analysis (10%)

- Check for demographic bias
- Evaluate fairness metrics
- Mitigate disparities

Part H: Report (10%)

- Executive summary for hospital administrators
- Technical methodology
- Clinical insights
- Implementation plan

4.6 Mini-Project Evaluation Rubric

Evaluation Rubric		
Criteria	**Weight**	**Description**
Data Preprocessing and EDA	15%	Quality of exploration, visualization, preprocessing
Baseline Models	10%	Implementation and evaluation of classical ML
Neural Network Architecture	20%	Creativity and appropriateness of architectures
Experimentation	15%	Thoroughness of hyperparameter tuning
Model Performance	15%	Final performance compared to baselines
Code Quality	5%	Code organization, comments, reproducibility
Visualizations	5%	Quality and clarity of visualizations
Interpretability	5%	Quality of model interpretation
Report Quality	10%	Clarity, organization, depth of analysis
Presentation	5%	Quality of oral presentation
Bonus: Deployment/Extra Features — +5%		

Submission Guidelines:

- Groups of 2-3 students
- Deadline: 3 weeks from assignment
- Submit via university platform
- Code must be runnable and documented
- Reports: max 15 pages PDF
- Presentations: 10-15 minutes (Optional, if the students and the lab instructor wish it)
- Choose a minimum of 2 projects; the group that completes more than 2 projects will be rewarded.

Summary of Datasets

Dataset	Task	Link
Iris	Multi-class Classification	UCI Link
Wine	Multi-class Classification	UCI Link
Breast Cancer	Binary Classification	UCI Link
Heart Disease	Binary Classification	UCI Link
Diabetes	Binary Classification	UCI Link
Wine Quality	Regression/Classification	UCI Link
Boston Housing	Regression	CMU Link
California Housing	Regression	Link
Adult Income	Binary Classification	UCI Link
Spambase	Binary Classification	UCI Link
Forest Covertype	Classification	UCI Link
Credit Card Fraud	Imbalanced Classification	Kaggle
Titanic	Binary Classification	Kaggle
Dogs vs Cats	Image Classification	Kaggle
Churn Modelling	Binary Classification	UCI Link
Telco Churn	Binary Classification	Kaggle
House Prices	Regression	Kaggle
Air Quality	Multi-output Regression	UCI Link
Energy Consumption	Time Series	UCI Link
Diabetes Readmission	Healthcare	UCI Link

Note: All datasets are freely available. Some Kaggle datasets require accepting terms and conditions.

"Practice is the best of all instructors."

— Publilius Syrus

Congratulations!

You've completed all exercises and mini-projects.
You now have practical experience with:

- Building neural networks with Keras
- Understanding key deep learning concepts
- Applying deep learning to real-world problems
- Developing comprehensive ML systems

"Practice is the best of all instructors."

— Publilius Syrus